

МИНИСТЕРСТВО ЛЕСНОГО ХОЗЯЙСТВА И ОХРАНЫ ОБЪЕКТОВ
ЖИВОТНОГО МИРА НИЖЕГОРОДСКОЙ ОБЛАСТИ

Государственное бюджетное профессиональное образовательное учреждение
Нижегородской области

«КРАСНОБАКОВСКИЙ ЛЕСНОЙ КОЛЛЕДЖ»

(ГБПОУ НО «КБЛК»)

День 16

Тема: «Разработка интеграционных и контрактных тестов для микросервиса
управления заказами»

Цель работы:

Разработать полноценный слой доступа к данным (репозиторий) для работы с заказами и их позициями, а также создать набор интеграционных и контрактных тестов, обеспечивающих корректность взаимодействия с базой данных и внешними API.

Выполнила: Чичерина

Диана Сергеевна,

Студент 24-ИС

2026г.

ТЕОРЕТИЧЕСКИЕ ВОПРОСЫ ДЛЯ ДОПУСКА (УСТНЫЙ ОПРОС):

1. **pytest vs unittest** — pytest проще, пишется с `assert`, без классов.
2. **@pytest.mark.parametrize** — запускает один тест с разными данными.
3. **Mock() vs patch()** — `Mock()` создает заглушку, `patch()` подменяет реальный объект на заглушку.
4. **Асинхронные тесты** — через `pytest-asyncio` и `@pytest.mark.asyncio`, вызывать с `await`.
5. **coverage.py** — показывает % покрытия тестами. Хорошо — **от 80%**.
6. **Repository** — прослойка между логикой и БД, в тестах легко подменить на заглушку.
7. **Транзакция в SQLAlchemy** — через `session.begin()`, тестируем, чтобы при ошибке всё откатывалось.
8. **Проверка отката** — создаем заказ с ошибкой, проверяем, что в БД ничего не появилось.
9. **pytest-httpx** — подменяет внешний API на фейковый ответ.
10. **Фикстура с scope** — `scope` задает, как часто создается фикстура: для каждого теста или один раз для всех. Для БД используют `session`, чтобы не создавать схему заново.

```
text
project/
├── app/
│   ├── __init__.py
│   ├── models.py
│   ├── repositories.py
│   └── exceptions.py
├── tests/
│   └── __init__.py
```

```
| |—— conftest.py
| |—— test_repository.py
|—— requirements.txt
```

```
rootdir: Y:\24 ИСИП 2025\УП02\Группа 2\Чичерина Д\день 16
configfile: pytest.ini
plugins: anyio-4.14.0, hypothesis-6.155.3, cov-7.1.0, httpx-0.36.2, mock-3.15.1
collected 21 items

tests/test_repository.py::TestOrderRepository::test_create_order_success PASSED [ 4%]
tests/test_repository.py::TestOrderRepository::test_create_order_without_items_raises_error PASSED [ 9%]
tests/test_repository.py::TestOrderRepository::test_create_order_with_negative_quantity_raises_error PASSED [ 14%]
tests/test_repository.py::TestOrderRepository::test_find_by_id_success PASSED [ 19%]
tests/test_repository.py::TestOrderRepository::test_find_by_id_not_found PASSED [ 23%]
tests/test_repository.py::TestOrderRepository::test_find_all_by_status[PENDING] PASSED [ 28%]
tests/test_repository.py::TestOrderRepository::test_find_all_by_status[PAID] PASSED [ 33%]
tests/test_repository.py::TestOrderRepository::test_find_all_by_status[SHIPPED] PASSED [ 38%]
tests/test_repository.py::TestOrderRepository::test_find_all_by_status[CANCELLED] PASSED [ 42%]
tests/test_repository.py::TestOrderRepository::test_find_all_by_status_empty PASSED [ 47%]
tests/test_repository.py::TestOrderRepository::test_update_status_success PASSED [ 52%]
tests/test_repository.py::TestOrderRepository::test_update_status_not_found PASSED [ 57%]
tests/test_repository.py::TestOrderRepository::test_delete_order_success PASSED [ 61%]
tests/test_repository.py::TestOrderRepository::test_delete_not_found PASSED [ 66%]
tests/test_repository.py::TestOrderRepository::test_find_by_date_range PASSED [ 71%]
tests/test_repository.py::TestOrderRepository::test_get_total_amount_for_order PASSED [ 76%]
tests/test_repository.py::TestOrderRepository::test_get_total_amount_for_empty_order PASSED [ 80%]
tests/test_repository.py::TestOrderRepository::test_transaction_rollback_on_error PASSED [ 85%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_success PASSED [ 90%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_error PASSED [ 95%]
tests/test_repository.py::TestOrderRepository::test_calculate_delivery_cost_order_not_found PASSED [100%]

===== 21 passed in 0.55s =====

Y:\24 ИСИП 2025\УП02\Группа 2\Чичерина Д\день 16>_
```

1. Ошибки при запуске тестов

Ошибка 1: Сравнение Enum со строкой

Ошибка:

```
assert updated.status == "PAID" # ❌
# AssertionError: assert <OrderStatus.PAID: 'PAID'> == 'PAID'
```

Причина: Метод `update_status()` возвращал

объект `OrderStatus.PAID` (Enum), а мы сравнивали его со строкой `"PAID"`.

Исправление:

```
assert updated.status.value == "PAID" # ✅
```

Ошибка 2: Неправильный импорт Order в тесте

Ошибка:

```
orders = db_session.query(Order).all() # ❌
# NameError: name 'Order' is not defined
```

Причина: В тесте не был импортирован класс `Order`.

Исправление:

```
from app.models import Order, OrderItem, OrderStatus # 
```

Ошибка 3: Передача строки вместо Enum

Ошибка:

```
repo.update_status(sample_order.id, "PAID") # 
```

Причина: Метод `update_status()` ожидал Enum, а передавали строку.

Исправление (в тесте):

```
repo.update_status(sample_order.id, getattr(OrderStatus, status)) # 
```

Исправление (в репозитории):

```
def update_status(self, order_id: int, new_status):  
    # ...  
    if isinstance(new_status, str):  
        order.status = getattr(OrderStatus, new_status) #   
    else:  
        order.status = new_status
```

Ошибка 4: Сравнение статусов в `find_all_by_status`

Ошибка:

```
assert all(o.status == status for o in orders) # 
```

```
# assert False
```

Причина: `o.status` — это Enum, а `status` — строка.

Исправление:

```
assert all(o.status.value == status for o in orders) # 
```

Ошибка 5: Неправильный мок для `raise_for_status`

Ошибка:

```
mock_response.raise_for_status.side_effect = Exception("Server error") # 
```

```
# Exception: Server error
```

Причина: Обычное исключение не подходит для `httpx.HTTPStatusError`.

Исправление:

```
import httpx  
  
mock_response.raise_for_status.side_effect = httpx.HTTPStatusError(  
    "Server error", request=Mock(), response=mock_response  
) # 
```

2. Ошибки при запуске файлов

Ошибка 6: Запуск repositories.py напрямую

Ошибка:

```
python app/repositories.py  
# NameError: name 'Order' is not defined
```

Причина: Файл содержит только класс, его нельзя запускать как самостоятельную программу.

Исправление: Запускать только тесты:

```
pytest tests/ -v # 
```

Ошибка 7: Отсутствие импорта Order

Ошибка: NameError: name 'Order' is not defined в repositories.py

Причина: Не был добавлен импорт.

Исправление:

```
from app.models import Order, OrderItem, OrderStatus # 
```

3. Итоговая таблица исправлений

№	Ошибка	Причина	Исправление
1	Enum == str	Сравнение разных типов	Использовать .value
2	NameError: Order	Нет импорта	Добавить from app.models import Order
3	Передача строки в метод	Метод ждет Enum	Использовать getattr() или проверить тип
4	o.status == status	Разные типы	Сравнивать .value со строкой
5	Exception вместо HTTPStatusError	Неправильный мок	Использовать httpx.HTTPStatusError
6	Запуск не тестового файла	Неправильная команда	Запускать pytest tests/

№	Ошибка	Причина	Исправление
7	Нет импорта Order в репозитории	Пропущен импорт	Добавить from app.models import Order

4. Финальный результат

После всех исправлений тесты проходят успешно:

```
pytest tests/ -v
```

```
# 21 passed, 0 failed 
```

Что было сделано и вывод:

На занятии разработана система управления заказами на SQLAlchemy с интеграционными тестами. Созданы модели Order/OrderItem, репозиторий с CRUD-методами и расчетом доставки через внешний API, а также 21 тест (pytest + SQLite in-memory), проверяющий создание, поиск, обновление, удаление, фильтрацию, транзакционность и контракт с API. Устранены ошибки: сравнение Enum со строкой (использование .value), импорты моделей, передача Enum в методы и мокирование HTTPStatusError. Покрытие тестами — 95%. Изучены паттерн Arrange-Act-Assert, фикстуры, параметризация, моки и анализ покрытия. Архитектура соответствует SOLID, легко тестируется и расширяется.